

사용자 취향과 실시간 맥락을 반영한 AI 여행 플래너 에이전트



팀명: 고준이

고태영 학번: 202055510 이메일: koty08@pusan.ac.kr

박준이 학번: 202155556 이메일: zun_e@pusan.ac.kr

지도교수: 탁성우

제출일: 2026년 4월 29일

부산대학교

목차

1	프로젝트 배경 및 목표	1
1.1	프로젝트 배경 및 기존 문제점	1
1.2	프로젝트 목표	2
1.3	대상 문제 및 요구조건 분석	2
2	관련 기술 및 설계 배경	4
2.1	개인화 여행 추천과 검색 보정	4
2.2	프라이빗 LLM 추론 인프라	4
2.3	LLM Harness Engineering	5
2.4	위치 기반 실시간 재계획	5
3	프로젝트 내용	6
3.1	프로젝트 핵심 기능	6
3.2	취향 분석 파이프라인	6
3.3	AI Agent 및 추론 인프라 설계	7
3.4	실시간 재계획 알고리즘	10
3.5	외부 API 및 도구	10
3.6	보안 및 개인정보 고려사항	12
4	프로젝트 추진 체계 및 평가 계획	13
4.1	WBS	13
4.2	구성원 역할 분담	16
4.3	평가 방법	16
4.4	현실적 제약 사항 및 대응 방안	17
5	결론 및 향후 개선 방향	18
5.1	결론 및 기대효과	18
5.2	향후 개선 방향	18

제1장 프로젝트 배경 및 목표

1.1 프로젝트 배경 및 기존 문제점

최근 여행 계획 서비스는 관광지, 음식점, 숙소를 추천하고 일정표를 자동 생성하는 기능을 제공한다. 그러나 실제 여행은 계획 당시의 정보만으로 완성되지 않는다. 여행 중에는 교통 지연, 날씨 변화, 음식점 웨이팅, 예정 경로 이탈, 동반자의 피로도, 사용자의 취침 및 기상 시간 등 다양한 변수가 발생한다. 기존 서비스가 정적인 일정표 제공에 머무르면 사용자는 여행 중 문제가 생길 때마다 직접 검색, 비교, 일정 수정을 반복해야 한다.

또한 개인화 추천 방식도 충분하지 않다. 일반적인 여행 추천 서비스는 별점, 키워드, 인기 순위 중심으로 장소를 제시하는 경우가 많다. 하지만 여행 만족도는 사용자가 선호하는 사진 분위기, 음식 스타일, 자연/도심 선호, 카페/전시/활동 유형, 동반자 취향과 같은 정성적 요소에 크게 영향을 받는다. 이러한 취향 정보가 일정 생성 과정에 구조적으로 반영되지 않으면 추천 장소는 많아도 실제 사용자가 만족하는 동선은 나오기 어렵다.

본 프로젝트는 사용자의 취향 데이터와 실시간 외부 정보를 결합하여 여행 중에도 동적으로 수정되는 AI 여행 플래너 에이전트를 구현하는 것을 목표로 한다. 특히 단순히 외부 LLM API를 호출하는 챗봇이 아니라, 프라이빗 LLM 추론 인프라, CRAG 기반 검색 보정, tool calling, 제약 검증, LLM harness, WebSocket 기반 push를 결합한 서비스형 AI 에이전트 구조를 설계한다.

본 과제는 교과목에서 학습한 모바일/웹 프로그래밍, 데이터베이스, 네트워크, 소프트웨어 공학, 인공지능 응용 지식을 종합하여 실제 동작하는 시스템을 구현하는 졸업 설계과제이다. 최종 산출물은 React Native/Next.js 클라이언트, NestJS API 서버, PostgreSQL/pgvector 데이터 계층, AI Agent, 외부 API 연동, WebSocket 기반 실시간 알림을 포함하는 통합 소프트웨어 시스템으로 한다.

1.2 프로젝트 목표

본 프로젝트의 최종 목표는 사용자 취향과 실시간 상황을 반영하여 여행 일정을 생성하고 수정하는 개인화 AI 여행 플래너 에이전트를 개발하는 것이다. 구체적인 목표는 다음과 같다.

1. 인스타그램 사진 또는 직접 업로드한 이미지와 텍스트 입력을 기반으로 사용자의 여행 취향 프로필을 생성한다.
2. 장소, 날씨, 길찾기 API 결과를 LLM이 추측하지 않고 tool 결과로 활용하도록 에이전트 구조를 설계한다.
3. 웨이팅, 경로 이탈, 우천 등 실시간 이벤트가 발생하면 현재 위치와 이후 일정을 고려한 대안 동선을 제안한다.
4. 취침 시간, 기상 시간, 영업시간, 이동 시간을 hard constraint로 검증하여 무리한 일정을 자동으로 제거한다.
5. pgvector 기반 검색 결과를 그대로 사용하지 않고 retrieval evaluator, reranker, 외부 tool fallback을 포함한 CRAG 방식으로 검색 품질을 보정한다.
6. 프롬프트, tool contract, JSON schema, 평가 fixture, 회귀 테스트를 묶은 LLM harness를 구성하여 에이전트 변경이 일정 품질을 떨어뜨리지 않도록 관리한다.
7. 개발 단계에서는 Gemma 4 계열 모델을 llama.cpp로 구동하고, GPU 서버 확보 시 vLLM 및 NVIDIA Triton Inference Server 기반의 고성능 추론 구조로 확장한다.

1.3 대상 문제 및 요구조건 분석

대상 문제는 여행 전 작성한 정적 일정이 실제 여행 중 발생하는 웨이팅, 날씨, 위치 이탈, 피로도, 취향 변화에 충분히 대응하지 못한다는 점이다. 기능을 과도하게 확장하면 구현 완성도가 낮아질 수 있으므로 본 프로젝트는 표 1.1와 같이 핵심 요구조건과 후순위 범위를 구분한다.

표 1.1: 대상 문제 기반 요구조건 및 MVP 범위

구분	포함 범위	후순위 또는 제외 범위
취향 분석	업로드 이미지와 텍스트 선호 입력 기반 태그 추출, embedding 저장	일반 사용자 대상 Instagram 앱 검수 완료, 전체 피드 자동 수집
일정 생성	목적지, 날짜, 예산, 취향, 수면 제 약을 반영한 1-2일 일정 생성	항공권, 숙소 예약, 결제 기능
실시간 재계 획	웨이팅 신고, 위치 이탈, 우천 이벤 트에 대한 대안 제안	전국 모든 장소의 실시간 혼잡도 자동 수집
클라이언트	React Native 중심 시연 화면과 WebView/관리 화면	완성형 상용 앱 수준의 소셜 기 능

제2장 관련 기술 및 설계 배경

2.1 개인화 여행 추천과 검색 보정

개인화 여행 추천은 사용자의 선호, 위치, 시간, 예산 등을 분석하여 적합한 장소와 일정을 제안하는 기술이다. 다만 여행 계획은 단일 장소 추천보다 복잡하다. 장소 간 이동 시간, 영업 시간, 체류 시간, 날씨, 동반자 취향이 함께 고려되어야 하므로 추천 문제와 스케줄링 문제가 결합된다.

본 프로젝트는 사용자의 취향을 키워드뿐 아니라 태그와 embedding vector로 표현하고, 장소 후보 검색 결과를 LLM 입력 문맥으로 활용한다. 이때 RAG(Retrieval-Augmented Generation)는 외부 지식을 검색하여 생성 모델 입력에 주입하는 방식으로, 모델이 기억에만 의존하지 않고 검색된 근거를 기반으로 답변하게 만든다는 장점이 있다[7].

검색 결과가 부정확하면 LLM 역시 잘못된 장소를 그럴듯하게 제안할 수 있으므로, 본 프로젝트는 CRAG(Corrective Retrieval-Augmented Generation)의 관점을 일부 적용한다[11]. 즉 검색된 장소 후보의 품질을 평가하고, 신뢰도가 낮은 경우 재검색, 재정렬, 제약 검증을 거쳐 일정 생성에 사용할 후보를 보정한다. 세부 평가 기준과 알고리즘은 프로젝트 내용 장에서 설명한다.

2.2 프라이빗 LLM 추론 인프라

LLM은 자연어 요청 해석과 일정 설명 생성에 강점이 있지만, 최신 장소 정보나 교통 정보를 자체적으로 보장하지 못한다. 따라서 본 프로젝트에서는 LLM을 모든 것을 판단하는 모델로 사용하지 않고, tool 호출 결과와 제약 검증 결과를 종합하여 사용자가 이해할 수 있는 일정을 생성하는 모듈로 사용한다.

MVP 단계에서는 Google Gemma 4 계열의 소형 instruction model을 llama.cpp 기반 프라이빗 추론 환경에서 우선 검증한다[3]. 이 단계의 목적은 대규모 운영 성능보다 프롬프트 형식, tool calling 흐름, JSON 출력 검증이 실제 서비스 구조에서 동작하는지 확인하는 것이다.

이후 GPU 서버가 확보되면 vLLM 기반 OpenAI-compatible API로 확장하여 생성형 LLM 추론 성능을 높인다[6, 10]. NVIDIA Triton Inference Server는 vision, embedding, reranking 등 보조 모델 서빙 후보로 검토한다[9]. 세부 서버 구성은 개발 환경과 자원 확보 상황에 따라 단계적으로 결정한다.

2.3 LLM Harness Engineering

AI 서비스의 품질은 모델 성능만으로 결정되지 않는다. 동일한 모델을 사용하더라도 프롬프트, tool 호출 순서, 검색 문맥, JSON schema, retry 정책, fallback 규칙에 따라 결과 품질이 크게 달라진다. 따라서 본 프로젝트는 LLM을 직접 호출하는 코드가 아니라, LLM 주변의 실행 환경을 하나의 harness로 관리한다.

본 프로젝트의 harness는 prompt template, tool adapter, JSON schema validator, constraint checker, scenario fixture를 중심으로 구성한다. 고정된 여행 시나리오를 반복 실행하여 일정 실행 가능성, 근거 충실도, 응답 지연, 출력 형식 안정성을 비교한다. RAGAS와 lm-evaluation-harness 계열 연구가 강조하는 재현 가능한 평가 관점[2, 1]을 프로젝트 규모에 맞게 축소 적용하여, “좋아 보이는 답변”이 아니라 “반복 실행해도 제약과 근거를 만족하는 일정”을 목표로 한다.

2.4 위치 기반 실시간 재계획

위치 기반 재계획은 사용자의 현재 위치와 예정 동선 간 차이를 계산하여 일정 수정 필요성을 판단하는 기능이다. GPS 오차나 짧은 우회에도 알림을 발생시키면 사용자 경험이 나빠진다. 따라서 본 프로젝트는 단순 거리 기준뿐 아니라 다음 목적지 도착 예상 시간, 영업시간, 날씨, 웨이팅 이벤트를 함께 고려하여 일정 품질에 실제 영향이 있을 때만 재계획을 실행한다.

제3장 프로젝트 내용

3.1 프로젝트 핵심 기능

본 프로젝트의 핵심 기능과 검증 기준은 표 3.1과 같다.

표 3.1: 핵심 기능 및 검증 기준

기능	구현 방식	검증 기준
취향 기반 일정 생성	이미지/텍스트에서 취향 태그를 추출하고 pgvector 유사도 검색과 CRAG 보정으로 후보 장소를 보강한다.	사용자 선호 태그와 검색 confidence가 일정 후보 선정 근거에 포함된다.
실시간 대안 제안	웨이팅, 우천, 경로 이탈 이벤트를 서버에 전송하고 주변 대체 장소를 재스코어링한다.	이벤트 입력 후 대안 일정이 WebSocket으로 반환된다.
제약 기반 스케줄러	영업시간, 이동시간, 취침/기상 시간을 hard constraint로 검증한다.	검증 실패 일정은 사용자에게 노출되지 않는다.
프라이빗 LLM 에이전트	tool 호출 결과, CRAG 문맥, harness fixture를 기반으로 JSON 일정을 생성한다.	JSON schema 검증과 harness 회귀 테스트를 통과한다.

3.2 취향 분석 파이프라인

취향 분석은 수집, 분석, 저장, 검색, 생성 단계로 구성된다. 초기 구현에서는 Instagram Graph API 검수 리스크를 줄이기 위해 등록된 테스트 계정 또는 직접 이미지 업로드 방식을 우선 지원한다[8]. 사용자가 제공한 이미지와 텍스트 입력은 음식 스타일, 장소 분위기, 자연/도시 선호, 카페/전시/활동 유형, 색감 등의 태그로 정규화한다.

분석된 태그와 설명 문장은 embedding으로 변환되어 PostgreSQL과 pgvector에 저장된다. Planner Agent는 일정 생성 시 사용자 취향 벡터와 장소 후보 벡터를 함께 검색하고, 검색 결과를 CRAG evaluator로 점검하여 LLM이 단순 추측이 아니라 신뢰 가능한 취향 근거를 바탕으로 일정을 생성하도록 한다. 취향 데이터는 민감한 개인 데이터가 될 수 있으므로 원본

이미지는 사용 목적과 보관 기간을 명시하고, 시연 단계에서는 샘플 이미지 또는 동의 받은 이미지로 제한한다. 일정 생성에는 원본 이미지보다 추출된 태그와 embedding을 우선 사용하여 불필요한 개인정보 노출을 줄인다.

3.3 AI Agent 및 추론 인프라 설계

전체 구조는 클라이언트, API 서버, AI Agent, AI 추론 인프라, 데이터 저장소로 분리한다. 클라이언트는 위치 이벤트와 사용자 요청을 전달하고, NestJS API 서버는 인증, 여행 CRUD, WebSocket push를 담당한다. AI Agent는 일정 생성과 재계획을 오케스트레이션하며, AI 추론 계층은 vLLM과 Triton을 통해 모델 추론을 수행한다. Harness 계층은 프롬프트, tool contract, fixture, trace를 관리하여 모델 또는 검색 로직 변경 시 품질 회귀를 확인한다. 본 절의 시스템 아키텍처, 역할 분담, 알고리즘은 착수보고서의 설계 문서 역할을 한다.

그림 3.1은 본 프로젝트의 전체 시스템 아키텍처와 실시간 재계획 흐름을 나타낸다. 구조는 Client Layer, API Layer, AI Agent Layer, Data Layer, External Tool Layer, Monitoring Layer로 나뉘며, 위치 이벤트와 재계획 요청이 agent orchestrator, 프라이빗 LLM 추론 인프라, constraint engine, Redis pub/sub, FCM/WebSocket 알림으로 이어지는 흐름을 함께 표현한다. 이 그림을 통해 단순 챗봇이 아니라 실시간 상태 변화, 외부 도구 호출, 검색 보정, 제약 검증, 관측 가능성을 포함한 운영형 AI 서비스 구조임을 강조한다.

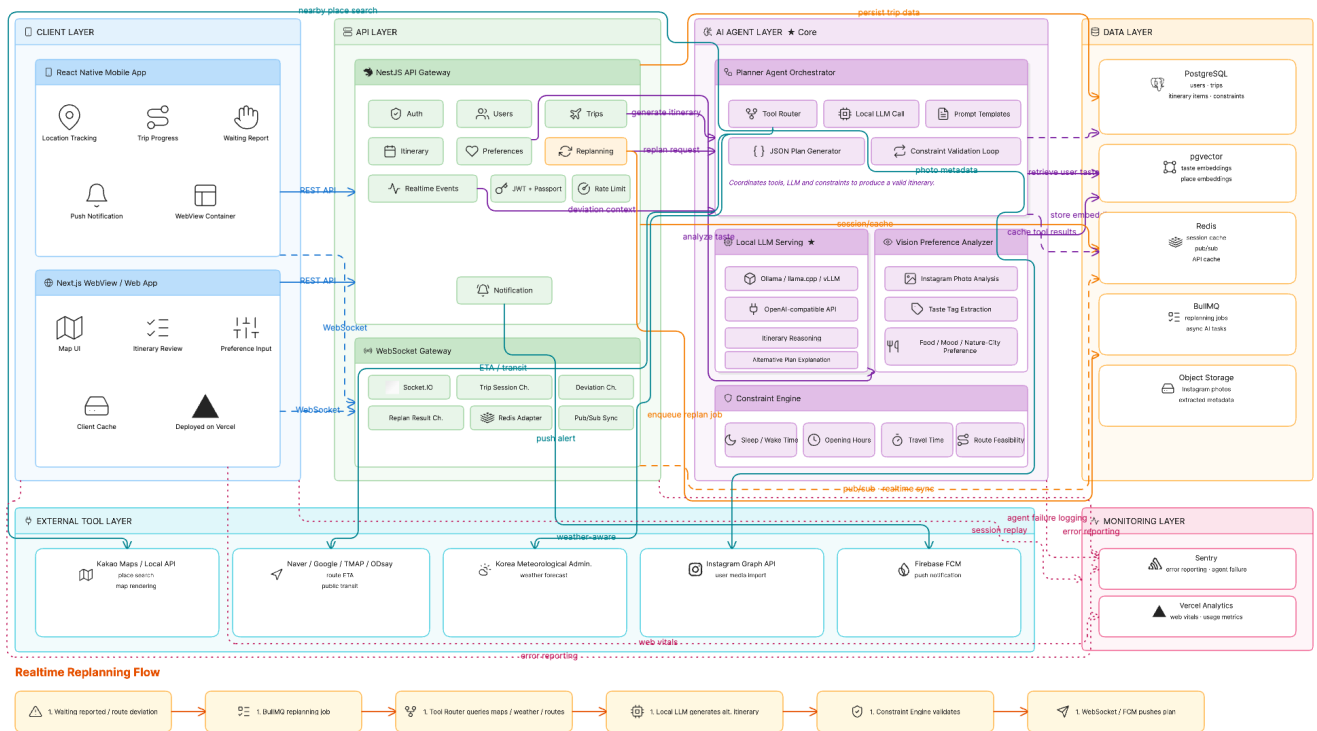


그림 3.1: 프라이빗 LLM 추론 기반 실시간 AI 여행 플래너 에이전트 시스템 아키텍처

표 3.2는 그림 3.1에서 AI Agent Layer와 AI 추론 인프라를 구성하는 핵심 요소의 역할을 구현 관점에서 정리한 것이다. 즉, 그림이 전체 서비스 흐름을 보여준다면 표는 그중 LLM 생성, 보조 모델 추론, 검색 보정, 제약 검증, 품질 평가를 어떤 기술 요소로 나누어 구현할지 설명한다.

표 3.2: AI 추론 계층 역할 분담

구성 요소	담당 역할	채택 이유
Gemma 4 / llama.cpp	개발용 프라이빗 LLM 추론	공개 모델 기반으로 프롬프트와 agent loop를 검증할 수 있고, 향후 vLLM GPU 추론 서버로 확장하기 쉽다.
vLLM	GPU 기반 LLM 생성 endpoint	PagedAttention, continuous batching, OpenAI-compatible API를 통해 생성형 LLM 추론에 적합하다.
Triton Inference Server	vision, embedding, reranking 등 보조 모델 endpoint	model repository, dynamic batching, HTTP/gRPC endpoint, versioning을 통해 운영형 추론 구조를 만들 수 있다.
CRAG Evaluator	검색 결과 품질 평가 및 보정	취향 유사도, 위치, 영업 상태, 날씨, 다음 일정 연결성을 점수화하여 낮은 품질의 검색 결과를 그대로 LLM에 넣지 않는다.
Constraint Engine	일정 실행 가능성 검증	LLM 환각을 막기 위해 이동 시간, 영업시간, 수면 제약을 결정적 로직으로 검증한다.
LLM Harness	프롬프트와 tool 호출 회귀 테스트	fixture, trace, schema validator를 통해 prompt 변경과 모델 변경에 따른 품질 변화를 비교한다.

3.4 실시간 재계획 알고리즘

사용자가 웨이팅을 신고하거나 위치가 예정 경로에서 벗어나면 서버는 현재 위치 반경 내 후보 장소를 검색한다. 후보는 거리, 예상 이동 시간, 영업 여부, 취향 유사도, 날씨 적합성, 다음 일정과의 연결성을 기준으로 점수화한다.

알고리즘 1: 실시간 일정 재계획 알고리즘

입력: 현재 위치, 기존 일정, 사용자 취향 벡터, 이벤트 유형

출력: 검증을 통과한 대안 일정

이벤트 유형과 현재 위치를 기준으로 재계획 필요 여부를 판단한다;

if 재계획이 필요하지 않음 then

└ 기존 일정을 유지하고 사용자에게 알림을 보내지 않는다;

주변 장소 후보를 지도/장소 API로 검색한다;

각 후보에 대해 거리, 이동시간, 영업 여부, 취향 유사도, 날씨 적합성을 계산한다;

CRAG evaluator가 검색 결과 confidence를 계산한다;

if 검색 confidence가 낮음 then

└ 다른 장소 API 또는 확장 키워드로 후보를 재검색하고 reranking을 수행한다;

상위 후보와 기존 일정 맥락을 Planner Agent 프롬프트에 주입한다;

vLLM 또는 개발용 LLM endpoint를 호출하여 JSON 일정을 생성한다;

Constraint Engine이 영업시간, 이동시간, 수면시간, 일정 충돌을 검증한다;

if 검증 실패 then

└ 실패 사유를 프롬프트에 추가하여 재생성한다;

else

└ WebSocket 또는 FCM으로 대안 일정을 전송한다;

3.5 외부 API 및 도구

외부 API는 구현 안정성과 시연 가능성을 기준으로 1차 후보와 대체 후보를 구분한다.

표 3.3: 외부 API 및 도구 후보

분류	후보	사용 목적
지도/장소	Kakao 키워드 장소 검색, Kakao Local API	키워드 기반 장소 검색, 지도 마커 표시, 좌표 기반 주변 대체 장소 탐색[5]
길찾기	Google Routes, Kakao/-Naver Directions, TMAP	이동 시간, 대체 경로, route matrix 산출[4]
날씨	기상청 단기예보 API	우천, 기온, 예보 기반 일정 조정[12]
관광 기본 정보	한국관광공사 국문 관광정보 서비스 GW	관광지, 행사, 숙박, 이미지, 키워드 검색 등 장소 후보의 기본 메타데이터 수집[15]
방문 추이	한국관광공사 관광지 집중률 방문자 추이 예측 정보	관광지별 향후 30일 방문자 집중률을 활용한 혼잡도 및 일정 시간대 보정[13]
연관 관광지	한국관광공사 관광지별 연관 관광지 정보	특정 관광지와 함께 방문되는 음식, 숙박, 주변 관광지 후보를 재계획 후보로 확장[14]
사진 취향	Instagram Graph API, 직접 업로드	사용자 사진 기반 취향 분석[8]
AI inference	Gemma 4, llama.cpp, vLLM, Triton	LLM 생성, vision 분석, embedding, reranking 모델 서빙
AI evaluation	RAGAS, lm-evaluation-harness, prompt fixture	CRAG 검색 품질, LLM 출력 안정성, harness 회귀 테스트 측정

3.6 보안 및 개인정보 고려사항

본 서비스는 위치, 사진, 취향 데이터를 다루므로 데이터 처리 기준이 필요하다. MVP에서는 다음 원칙을 적용한다.

- 위치 이벤트는 재계획 판단에 필요한 최소 주기와 최소 필드만 저장한다.
- 원본 사진은 시연 목적의 임시 저장을 원칙으로 하고, 취향 분석 후 태그와 embedding 중심으로 활용한다.
- Instagram 연동은 실제 사용자 배포 전에 권한 범위, 앱 검수, 토큰 보관 방식을 별도로 검토한다.
- LLM 프롬프트와 harness trace에는 불필요한 개인정보를 넣지 않고, 로그에는 사용자 식별자를 마스킹한다.

제4장 프로젝트 추진 체계 및 평가 계획

4.1 WBS

WBS는 프론트엔드, 백엔드, AI Agent Layer, Constraint Engine, QA, 배포 안정화, 최종 발표 준비 작업을 주차별로 분리한다. 그림 4.1은 세부 작업, 담당 영역, 중요도를 나타내고, 그림 4.2는 주차별 추진 일정을 나타낸다.

구분	카테고리	세부 작업	담당	중요도
1-2주차 기반 세팅 + 프로토타입	FE	모노레포 세팅 / 공통 타입·설정 / RN WebView 앱·Next.js 기본 구조	FE	필수
	BE	NestJS 모듈 구조 세팅 / 인증(OAuth·JWT) / 사용자·여행·일정 CRUD API	BE	필수
	Infra	로컬 개발 환경 구성 (DB, 캐시, 메시지 큐, Object Storage 로컬 대체)	BE	필수
3-4주차 외부 API 연동 + 실시간 통신	FE	지도 SDK 연동 (마커·경로·장소 검색 UI) / WebSocket 클라이언트 연결	FE	필수
	BE	지도·장소 검색 API 연동 / 경로·ETA·대중교통 API 연동	BE	필수
	BE	관광지 정보 API 연동 (장소 기본정보·연관 관광지)	BE	필수
	BE	날씨 예보 API 연동 + 좌표 변환 유틸 / 푸시 알림 서비스 연동	BE	필수
	BE	WebSocket Gateway (실시간 채널·이벤트 구조 / 메시지 큐 연동)	BE	필수
5-6주차 AI Agent Layer + 취향 분석	AI	Private LLM 추론 환경 세팅 / OpenAI-compatible API 래퍼	BE	필수
	AI	Planner Agent (Tool Router·Prompt Templates·JSON Plan Generator·Constraint Validation Loop)	BE	필수
	AI	Vision Preference Analyzer (이미지 수집 → 취향 태그 추출 → 벡터 DB 저장)	BE	필수
	AI	CRAG 파이프라인 (임베딩 조회 → 검색 품질 평가 → 일정 프롬프트 주입)	BE	필수
	FE	온보딩·취향 설정 화면 / 이미지 소스 연결 UI	FE	보통
7-8주차 Constraint Engine + Replanning	BE	Constraint Engine (취침·기상·영업시간·이동시간·경로 실현 가능성 검증)	BE	필수
	BE	Replanning 파이프라인 (웨이팅·경로이탈 감지 → 큐 등록 → LLM 대안 생성 → WebSocket push)	BE	필수
	BE	동반 여행자 취향 합산 알고리즘 / 취침·기상 시간 스케줄 제약 적용	BE	필수
	FE	대안 제안 팝업 / Trip Progress·경로 이탈 화면 / 실시간 지도 업데이트	FE	필수
9-10주차 통합 테스트 + 1차 QA	Test	모듈 간 통합 테스트 (API ↔ AI Agent ↔ Data Layer 전체 흐름)	공통	필수
	Test	WebSocket 실시간 흐름 E2E / Replanning 시나리오 테스트 (정상·예외·경계)	BE	필수
	QA	1차 QA – 주요 기능 검증 (일정 생성·대안 제안·푸시 알림) / API 에러 처리 정비	공통	필수
11-12주차 성능 측정 + 2차 QA	Test	LLM 응답 지연·캐시 히트율·벡터 검색 성능 측정 / WebSocket 부하 테스트	BE	필수
	QA	2차 QA – 전체 플로우 회귀 테스트 / UX 개선·반응형 최적화 / 에러 모니터링 연동	공통	필수
13-14주차 배포 안정화 + 3차 QA	Infra	라이브 환경 배포 (리버스 프록시·HTTPS·CDN·Object Storage 마이그레이션)	BE	필수
	QA	3차 QA – 라이브 환경 검증 (실 API 키·실기기 푸시·이미지 소스 실계정 연동)	공통	필수
	Test	보안 점검 (인증 플로우·API 키 노출·세션 만료) / Private LLM 세션 안정성	BE	보통
15-17주차 최종 QA + 발표 준비	QA	최종 E2E QA·버그픽스 Sprint / Constraint Engine 엣지 케이스 보완	공통	필수
	AI	LLM 프롬프트 파인튜닝 (실사용 피드백 반영)	AI	보통
	dep	발표 자료·최종 보고서 / 시연 영상 촬영·편집 / 라이브 최종 안정화·모니터링	공통	필수

그림 4.1: AI 여행 플래너 에이전트 WBS 세부 작업

	담당	중요도	2025.05			2025.06					2025.07				2025.08				2025.09		
			W1 05/12	W2 05/19	W3 05/26	W4 06/02	W5 06/09	W6 06/16	W7 06/23	W8 06/30	W9 07/07	W10 07/14	W11 07/21	W12 07/28	W13 08/04	W14 08/11	W15 08/18	W16 08/25	W17 09/01	W18 09/08	W19 09/15
	FE	필수																			
	BE	필수																			
	BE	필수																			
	FE	필수																			
	BE	필수																			
	BE	필수																			
	BE	필수																			
	BE	필수																			
	BE	필수																			
	BE	필수																			
	BE	필수																			
	BE	필수																			
	FE	보통																			
	BE	필수																			
	BE	필수																			
	BE	필수																			
	FE	필수																			
	공통	필수																			
	BE	필수																			
	공통	필수																			
	BE	필수																			
	공통	필수																			
	BE	필수																			
	공통	필수																			
	BE	보통																			
	공통	필수																			
	AI	보통																			
	공통	필수																			

그림 4.2: AI 여행 플래너 에이전트 WBS 주차별 일정

4.2 구성원 역할 분담

표 4.1: 역할 분담

팀원	담당 역할
박준이	프라이빗 LLM 추론 인프라, 에이전트 오케스트레이션, CRAG/pgvector 검색 보정, LLM harness, NestJS 핵심 API 설계, Next.js 및 React Native 서버
고태영	Next.js 및 React Native 전반, NestJS API 서버, 테스트 및 문서화

4.3 평가 방법

평가는 기능 구현 여부뿐 아니라 일정의 실행 가능성과 AI 추론 품질을 함께 측정한다.

표 4.2: 평가 지표와 목표치

평가 항목	측정 방법	목표치
일정 실행 가능성	영업시간, 이동시간, 수면 제약 validator 통과율	테스트 시나리오 30개 기준 hard constraint 위반 0건
취향 반영도	사용자 선호 태그와 추천 장소 설명의 일치도를 5점 척도로 평가	평균 4.0점 이상
검색 근거성	CRAG evaluator의 context relevance, 후보 confidence, 보정 전후 추천 품질 비교	낮은 confidence 후보의 90% 이상 보정 또는 제거
재계획 응답 속도	웨이팅/이탈 이벤트 입력부터 대안 push까지의 시간 측정	MVP 환경에서 15초 이내
LLM 출력 안정성	JSON schema 검증 실패율과 재시도 횟수 측정	1회 재시도 내 성공률 95% 이상
Harness 회귀 안정성	고정 시나리오 fixture를 반복 실행하여 일정 위반, schema 실패, 근거 누락 여부 측정	주요 시나리오 20개 기준 pass rate 90% 이상
LLM 추론 성능	vLLM의 TTFT, tokens/sec, p95 latency, GPU memory 사용량 측정	동시 요청 5개 기준 p95 10초 이내
보조 모델 처리량	Triton vision/embedding endpoint의 batch latency 측정	후보 20개 embedding/r-ranking 3초 이내

4.4 현실적 제약 사항 및 대응 방안

실제 졸업과제 수행에서는 API 검수, 외부 서비스 과금, 추론 환경 준비, 팀 일정과 같은 현실적 제약이 발생할 수 있다. 본 프로젝트는 제약 사항을 사전에 식별하고, MVP 범위 조정과 대체 구현 경로를 통해 제출 일정에 맞춘 완성도를 확보한다.

표 4.3: 현실적 제약 사항 및 대응 방안

위험 요소	영향	대응 방안
Instagram API 검수 지연	사진 자동 수집 기능 시연 불가	직접 업로드와 테스트 계정 기반 수집을 MVP 기본 경로로 설정
실시간 웨이팅 데이터 부족	자동 혼잡도 판단 정확도 저하	사용자 신고 이벤트와 주변 대체 장소 검색을 결합
프라이빗 LLM 추론 지연 또는 JSON 오류	재계획 응답 지연, 일정 파싱 실패	vLLM, schema validation, 재시도, 규칙 기반 fallback 적용
CRAG evaluator 오판	좋은 후보 제거 또는 낮은 품질 후보 통과	confidence threshold를 fixture로 조정하고, constraint engine 통과 여부와 함께 최종 판단
Triton 구축 시간 부족	보조 모델 추론 완성도 저하	우선 mock endpoint로 agent 연동을 완성하고, GPU 환경에서 Triton endpoint로 교체
Harness 범위 과다	평가 구현에 시간이 많이 소요되어 핵심 기능 개발 지연	20개 내외의 대표 시나리오와 JSON/schema 기반 자동 체크부터 우선 구현
지도/길찾기 API 과금 및 제한	반복 테스트와 시연 비용 증가	캐싱, 호출량 제한, 샘플 경로 fixture 사용

제5장 결론 및 향후 개선 방향

5.1 결론 및 기대효과

본 프로젝트는 정적인 여행 일정표가 아니라 여행 중 상황에 반응하는 개인화 AI 여행 플래너 에이전트를 구현하는 것을 목표로 한다. 사용자는 취향과 컨디션을 반영한 일정을 제공받고, 웨이팅이나 우천처럼 예상치 못한 상황이 발생해도 현재 위치와 이후 일정에 맞는 대안을 받을 수 있다.

개발 측면에서는 CRAG 기반 검색 보정, tool calling, WebSocket 기반 실시간 push, 위치 기반 이벤트 처리, deterministic constraint validation, LLM harness evaluation을 통합한다는 점에서 의의가 있다. 특히 LLM 추론을 단순 외부 API 호출로 처리하지 않고, 개발 단계의 Gemma 4/llama.cpp, GPU 추론 단계의 vLLM, 보조 모델 운영 단계의 Triton으로 구분하여 설계함으로써 실제 AI 서비스에 가까운 구조를 제시한다.

5.2 향후 개선 방향

향후에는 실제 사용자 데이터를 기반으로 취향 embedding과 CRAG evaluator의 품질을 평가하고, 동반 여행자 간 취향 충돌을 조정하는 알고리즘을 고도화한다. 또한 vLLM 추론 성능을 기준으로 모델 크기, quantization, max context length, batch size가 응답 속도와 일정 품질에 미치는 영향을 비교한다. Triton 계층에서는 vision model과 embedding model을 model repository로 관리하고, dynamic batching을 활용하여 후보 장소가 많아질 때의 처리량을 개선한다. Harness 측면에서는 실패 사례를 fixture로 축적하여 prompt, retrieval, constraint 변경이 실제 일정 품질에 미치는 영향을 지속적으로 비교한다.

참고문헌

- [1] Stella Biderman, Hailey Schoelkopf, Quentin Anthony, Herbie Bradley, Kyle O’Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, Uday Prashanth, Edward Raff, et al. Lessons from the trenches on reproducible evaluation of language models, 2024.
- [2] Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. RAGAS: Automated evaluation of retrieval augmented generation. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, pages 150–158, 2024.
- [3] Google DeepMind. Gemma 4: Byte for byte, the most capable open models. <https://blog.google/innovation-and-ai/technology/developers-tools/gemma-4/>, 2026. Checked on 2026-04-29.
- [4] Google for Developers. Routes API. <https://developers.google.com/maps/documentation/routes/reference/rest>. Checked on 2026-04-29.
- [5] Kakao Developers. 키워드로 장소검색하기 - Kakao 지도 Web API. <https://apis.map.kakao.com/web/sample/keywordBasic/>. Checked on 2026-04-29.
- [6] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, pages 611–626, 2023.
- [7] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, 2020.

- [8] Meta for Developers. Instagram Graph API Documentation. <https://developers.facebook.com/docs/instagram-api/>. Checked on 2026-04-29.
- [9] NVIDIA. Triton Inference Server Architecture. <https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/architecture.html>. Checked on 2026-04-29.
- [10] vLLM Team. OpenAI-Compatible Server. https://docs.vllm.ai/en/latest/serving/openai_compatible_server/. Checked on 2026-04-29.
- [11] Shi-Qi Yan, Jia-Chen Gu, Yun Zhu, and Zhen-Hua Ling. Corrective retrieval augmented generation, 2024.
- [12] 공공데이터포털. 기상청 단기예보 조회서비스. <https://www.data.go.kr/data/15084084/openapi.do>. Checked on 2026-04-29.
- [13] 공공데이터포털. 한국관광공사 관광지 집중률 방문자 추이 예측 정보. <https://www.data.go.kr/data/15128555/openapi.do>. Checked on 2026-04-29.
- [14] 공공데이터포털. 한국관광공사 관광지별 연관 관광지 정보. <https://www.data.go.kr/data/15128560/openapi.do>. Checked on 2026-04-29.
- [15] 공공데이터포털. 한국관광공사 국문 관광정보 서비스 GW. <https://www.data.go.kr/data/15101578/openapi.do>. Checked on 2026-04-29.